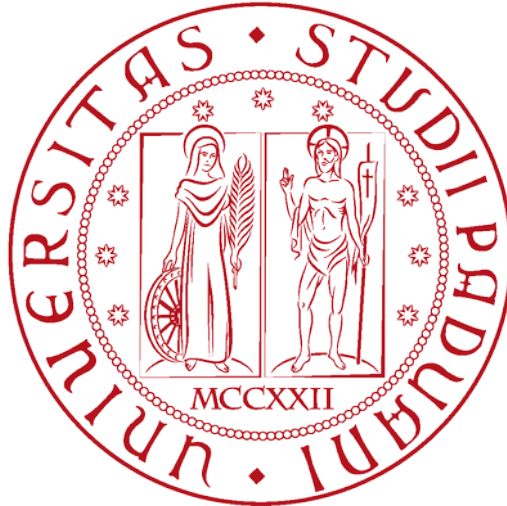


UNIVERSITY OF PADOVA

School of Engineering
Department of Information Engineering



Computer Vision Final Project Proposal
Visual Odometry for localization purposes

by

Bjørn Magnus Myrhaug - 2141821

Maximilian Pries - 2144162

Nihal Suri - 2141829

Submitted to

Prof. Stefano Ghidoni

Padova, July 23, 2025

Contents

1	Introduction	1
1.1	Activity goal	1
1.2	Current methodologies	1
2	Applications	2
3	Performance Measurement	3
4	Dataset	4
	References	X

1 Introduction

Visual Odometry (VO) is a technique that estimates the position and movement of a camera or vehicle by analyzing a sequence of images. It's a computer vision-based method that allows autonomous vehicles and robots to estimate their position without relying on external sensors like GPS, especially in environments with weak or no signal. VO works by tracking features in consecutive images and using them to calculate the relative poses between the frames.

1.1 Activity goal

The aim is to develop a C++/OpenCV system that estimates camera motion (position and orientation) from input images. Specifically:

- **Objective:** Given a sequence of images captured by a moving camera, compute the corresponding camera poses (trajectory) over time. This typically means estimating the 6-DoF transformation between successive frames.
- **Inputs:** Sequential images (monocular or stereo, possibly including depth information) from the vehicle's camera(s). The images should be calibrated (camera intrinsics known) and synchronized.
- **Outputs:** The estimated camera movement, usually a list of rototranslations from frame to frame (position and orientation) in a world coordinate frame, and from that, a reconstructed trajectory. Optionally, a sparse 3D point map of tracked features can be produced, but the primary output is the pose sequence.
- **Scope and Limitations:** We will focus on pure visual odometry (sequential pose estimation) to reconstruct only relative movement from a starting point. The main task consists of utilizing only monocular RGB or grayscale inputs, while the extension to stereo or the inclusion of depth information is optional. The system will not use additional sensors (IMU, GPS) or external mapping. **For monocular input, the scale of translation cannot be determined without extra information, which is a known limitation.**

1.2 Current methodologies

Visual odometry methods can be broadly categorized into classical (geometry-based) and deep learning approaches:

- **Feature-based VO pipelines:** Traditional VO follows a pipeline of feature detection, matching, and pose estimation. Keypoints are detected in each image (e.g., using FAST, ORB or SIFT) and descriptors are computed. Correspondences between frames are found by matching descriptors (e.g., with a brute-force or FLANN matcher). Given matches, the relative camera motion is estimated using PnP algorithms. For example, one can compute the essential

matrix and recover rotation/translation, or use `cv::solvePnP()` with RANSAC to estimate pose from 3D-2D correspondences. Optical flow methods (like Lucas-Kanade) can also track points between frames. OpenCV provides many of these building blocks: `cv::ORB` (feature detector), `cv::calcOpticalFlowPyrLK`, `cv::BFMatcher`, and `cv::recoverPose`, `cv::solvePnP` can be used to implement a complete VO pipeline in C++.

- **ORB-SLAM2 (feature SLAM):** As a practical example, ORB-SLAM2 is a widely used real-time SLAM system that extends VO by adding mapping and loop-closure, it uses ORB features for tracking monocular, stereo or RGB-D cameras. ORB-SLAM2 computes the camera trajectory and builds a sparse 3D map in real time (with true scale for stereo/RGB-D). It also detects loop closures and relocalizes if tracking is lost. We will reference ORB-SLAM2's approach as a state-of-the-art baseline, although our implementation will focus on the VO component (without full loop closing).
- **Deep learning approaches:** Recent VO research uses Convolutional Neural Networks (CNNs) and recurrent networks to estimate motion end-to-end. For instance, DeepVO (Wang et al. 2017) uses a CNN+RNN architecture to infer poses directly from image sequences without explicit feature matching. The network is trained end-to-end on image pairs to predict inter-frame pose.
- **OpenCV for VO:** OpenCV facilitates implementation of classical VO. In C++, one can use `cv::ORB` or `cv::FAST` for keypoint detection, `cv::BFMatcher` or `cv::FlannBasedMatcher` for descriptor matching, and `cv::calcOpticalFlowPyrLK()` for optical flow tracking. Pose estimation can be done with `cv::recoverPose()` (using essential matrix) or `cv::solvePnP()` (with known 3D points). OpenCV also supports stereo depth computation (`cv::StereoBM/SGBM`) which can aid stereo VO. By chaining these functions, we can build a working VO system in C++.

2 Applications

Visual odometry is critical in many domains:

- **Robotics and autonomous vehicles:** Robots and self-driving cars use VO to localize relative to their environment when GPS is unavailable or unreliable. For example, autonomous cars estimate their motion through streets using cameras and VO, often fusing with other sensors. Robots exploring unknown environments rely on VO for navigation and path planning.
- **Augmented/Virtual Reality (AR/VR):** In AR/VR headsets and mobile AR applications, visual odometry (often combined with IMU) tracks the user's viewpoint in real time. Accurate VO is crucial to overlay graphics consistently as the camera moves.

- **Structure-from-Motion and Mapping:** VO algorithms form the front-end of SLAM and SfM pipelines. Good VO reduces drift during mapping, improving overall localization accuracy.
- **Realtime Localization:** VO provides continuous pose updates at camera frame rates, making it suitable for real-time applications. Even without loop closures, VO is often used for “dead reckoning” between map alignments.

3 Performance Measurement

Standard metrics for evaluating VO accuracy include:

- **Absolute Trajectory Error (ATE):** The ATE measures the global difference between the estimated trajectory and the ground truth trajectory. After aligning the estimated path with the ground truth (usually by a rigid-body transformation), ATE computes the root-mean-square (RMS) distance between corresponding positions. In practice, one associates each estimated pose with the closest ground-truth pose in time and then calculates the positional deviations. An example of the same is shown in Figure 3.1.

$$\sqrt{\frac{1}{N} \sum_i \|\mathbf{p}_i^{est} - \mathbf{p}_i^{gt}\|^2}$$

where $\mathbf{p}_i^{est}$ and $\mathbf{p}_i^{gt}$ are the estimated and ground-truth positions.

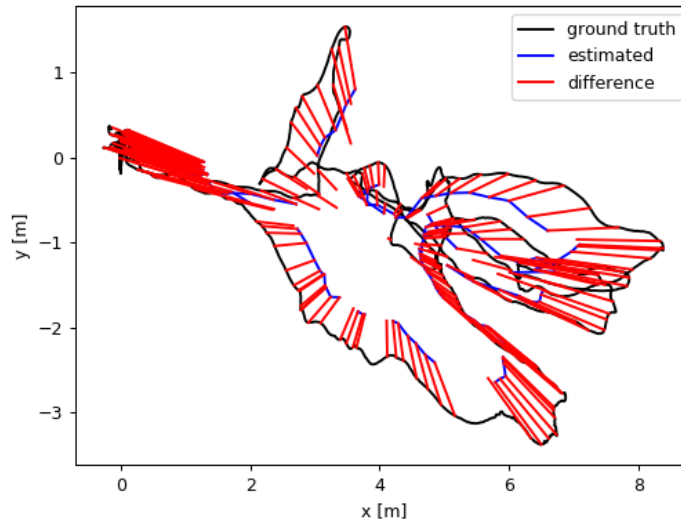


Figure 3.1: Example of Absolute Trajectory Error visualization

- **Relative Pose Error (RPE):** Relative pose error (RPE) in visual odometry measures the accuracy of estimated camera movements compared to the true

relative movements. It's used to evaluate the drift of a visual odometry system or the accuracy of loop closures in SLAM. RPE compares the estimated relative transformations between nearby poses with the actual relative transformations (ground truth).

The RPE can be measured as the error in relative transformations, e.g.

$$\mathbf{T}_i^{-1}\mathbf{T}_{i+\Delta}$$

for a fixed Δ (often $\Delta = 1$ for consecutive frames) compared against ground truth. These metrics allow quantifying global accuracy (ATE) and drift (RPE) of the VO system.

4 Dataset

The considered dataset is a subset of the "RGB-D SLAM Dataset" provided in [1]. The subset contains the "Testing and Debugging" as well as the "Robot SLAM" categories. As the name suggests, the first one is meant to serve as a possibility for testing the designed algorithm on simple datasets with little motion blur, while the second one contains image sequences produced by a robot-mounted camera, steered through halls on closed-loop tracks. In this category, motion blur is present, and lighting conditions are not constant. Each sequence is provided in a `.tgz` folder and contains:

- a folder `depth` containing the `.png` files for the depth information encoded in grayscale
- a folder `rgb` containing the `.png` files of the image sequence in RGB format
- a file `accelerometer` containing accelerometer data with corresponding timestamps
- a file `depth` containing the timestamps for each depth grayscale file
- a file `rgb` containing the timestamps for each image file
- a file `groundtruth` containing the rototranslations between each frame with the corresponding timestamp

Further information can be found at the [TUM website](#), which provides the sequences. The authors also provide [tools for automated evaluation](#) of the ATE and RPE. The selected subset of the dataset can be downloaded from [Google Drive](#). Figures 4.1 to 4.6 show samples from the dataset, showcasing the different lighting conditions and motion blur.



Figure 4.1: Sample image 1 from Robot SLAM.



Figure 4.2: Sample image 2 from Robot SLAM.



Figure 4.3: Sample image 3 from Robot SLAM.



Figure 4.4: Sample image 4 from Robot SLAM.



Figure 4.5: Sample image 5 from Robot SLAM.



Figure 4.6: Sample image from Testing and Debugging.

References

- [1] STURM, J. ; ENGELHARD, N. ; ENDRES, F. ; BURGARD, W. ; CREMERS, D.
: A Benchmark for the Evaluation of RGB-D SLAM Systems. In: *Proc. of the International Conference on Intelligent Robot Systems (IROS)*, 2012